GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA

ASIGNATURA:	CONSTRUCCION DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	**Refactorizar hacia patrones de diseño con IA: Strategy, Adapter, Factory, Decorator, State, Observer y Chain of Responsibility, medidos con pruebas y métricas**				
NÚMERO DE PRÁCTICA:	07	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VI
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		3
FECHA INICIO:	19/10/2026	**FECHA FIN:**	24/10/2026	**DURACIÓN:**	90 min presenciales + trabajo autónomo

RECURSOS A UTILIZAR:

Computador con acceso a Internet; repositorio de los Laboratorios 04 a 06 en GitHub; Python 3.10+ con pytest, radon y pylint (incluye pyreverse); Visual Studio Code con la extensión PlantUML; PlantUML (Java 17+ o servidor web plantuml.com); refactoring.guru/es/design-patterns como referencia visual; asistente de IA generativa (Claude, ChatGPT, Gemini o GitHub Copilot); aula virtual del curso. No se requiere clave de API de ningún LLM: las pruebas usan modelos simulados.

DOCENTE(s):

Mg. Antonio Arroyo Paz

OBJETIVOS/TEMAS Y COMPETENCIAS

OBJETIVOS:

1. Reconocer olores de código y relacionarlos con el patrón de diseño GoF y el principio SOLID que los resuelven, justificando cuándo NO conviene aplicar un patrón.
2. Refactorizar de forma segura un flujo real hacia al menos cuatro patrones, protegiendo el comportamiento con pruebas de caracterización.
3. Aislar un componente de IA no determinista detrás de patrones (Strategy, Adapter, Decorator y Chain of Responsibility) para que el sistema sea probable y sustituible.
4. Medir el efecto del refactor con métricas objetivas (complejidad ciclomática, líneas de código) y documentarlo con diagramas como código, un ADR y la bitácora de IA.

TEMAS:

- Concepto de patrón de diseño y catálogo GoF: creacionales, estructurales y de comportamiento.
- Principios SOLID, "programar contra interfaces" y "composición sobre herencia".
- Patrones Strategy, Adapter, Factory, Decorator, State, Observer y Chain of Responsibility.
- Olores de código, refactorización hacia patrones y métricas (complejidad ciclomática con radon).
- Patrones para integrar componentes de IA: sustitución de proveedores, caché, reintentos y guardrails; riesgos del uso de IA para diseñar.

COMPETENCIAS	**C.b:** Genera de forma responsable prototipos, experimentos y modelos, con el fin de analizar e interpretar información para la toma de decisiones fundamentadas y objetivas.

	C.c: Diseña responsablemente sistemas, componentes o procesos para satisfacer necesidades dentro de restricciones realistas.
	C.d: Trabaja éticamente, de manera efectiva, en equipos multidisciplinarios, adaptándose a diferentes entornos y nuevas situaciones.
	C.g: Comunica con claridad y de forma efectiva ideas, resultados y productos relacionados con su ejercicio profesional.

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

1.1 ¿Qué es un patrón de diseño?

Un patrón de diseño es una **solución probada a un problema de diseño que se repite**, descrita de forma que pueda reutilizarse en contextos distintos. Gamma, Helm, Johnson y Vlissides (el "Gang of Four", GoF, 1994) catalogaron 23 patrones orientados a objetos y describieron cada uno con cuatro elementos: **nombre**, **problema**, **solución** y **consecuencias**. Las consecuencias importan tanto como la solución: todo patrón agrega clases e indirección, y solo se justifica si resuelve un problema real.

Los patrones existen en distintos niveles. En el **Laboratorio 04** se trabajaron patrones **arquitectónicos** (capas, monolito modular, eventos), que organizan el sistema completo. En este laboratorio se trabajan patrones de **diseño**, que organizan las clases **dentro de un módulo**, como los diseñados con UML en el Laboratorio 05.

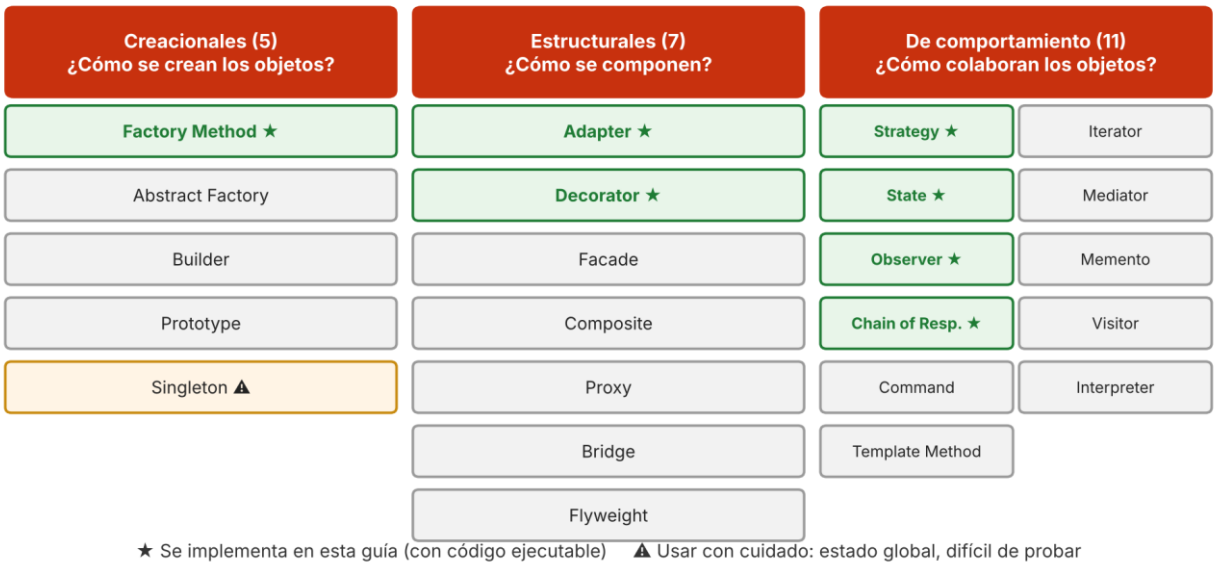


Figura 1. *Catálogo GoF de 23 patrones. Se marcan los siete que se implementan en esta guía. Gráfico elaborado con Python (matplotlib).*

1.2 Los principios detrás de los patrones

Los patrones no son el objetivo: son la forma concreta de aplicar principios de diseño. Los más importantes son **"programar contra interfaces, no contra implementaciones"**, **"preferir composición sobre herencia"** (GoF, 1994) y los principios **SOLID** (Martin, 2017):

Tabla 1. Principios SOLID y su relación con los patrones

Principio	Idea	Patrones que lo materializan
S — Responsabilidad única	Una clase debe tener una sola razón para cambiar.	Observer, Chain of Responsibility

O — Abierto/cerrado	Extender el comportamiento sin modificar el código existente.	Strategy, Factory, Decorator, Observer
L — Sustitución de Liskov	Toda implementación debe poder usarse donde se espera su interfaz.	Strategy, Adapter, State
I — Segregación de interfaces	Interfaces pequeñas y específicas, no "interfaces gordas".	Adapter (puertos pequeños)
D — Inversión de dependencias	Depender de abstracciones, no de detalles (SDK, APIs de terceros).	Adapter, Strategy, Factory

1.3 Los siete patrones de esta guía

Tabla 2. Patrones que se implementan en el ejercicio resuelto

Patrón	Tipo	Intención	Olor que resuelve	Ejemplo en AgroConecta
Strategy	Comport.	Encapsular algoritmos intercambiables detrás de una interfaz.	if/elif que elige "cómo hacer algo"	Cobrar con Yape o con Plin
Adapter	Estruct.	Convertir una interfaz ajena en la que espera el sistema.	Código del dominio que conoce detalles de un SDK	YapeSDK.realizar_cobro( ) → cobrar()
Factory Method (registro)	Creac.	Delegar la creación del objeto adecuado.	new/constructores repartidos por el código	FabricaPasarelas.crear(" YAPE")
Decorator	Estruct.	Agregar responsabilidades envolviendo un objeto.	Lógica transversal duplicada (reintentos, caché, log)	ConReintentos, ConCache
State	Comport.	Cambiar el comportamiento según el estado interno.	Condicionales sobre un atributo "estado"	Borrador, Confirmado, Pagado, Cancelado
Observer	Comport.	Notificar a interesados sin acoplarse a ellos.	Un método que "avisa a todos" a mano	WhatsApp y reportes al pagar
Chain of Responsibility	Comport.	Pasar una solicitud por una cadena de manejadores.	Validaciones en secuencia mezcladas	Guardrails de la salida del LLM

1.4 Olores de código y refactorización hacia patrones

Un **olor de código** (Fowler, 2018) es una señal superficial de un problema de diseño: métodos largos, condicionales que crecen con cada requisito, código duplicado o clases que conocen demasiado. Kerievsky (2004) propone **refactorizar hacia patrones**: no se diseña "con patrones" desde el inicio, sino que se introduce un patrón **cuando un olor concreto lo pide**, en pasos pequeños y protegidos por pruebas.

Para medir el efecto se usa la **complejidad ciclomática** (McCabe, 1976): el número de caminos independientes de un método. La herramienta **radon** la calcula para Python y la clasifica de A (1–5, simple) a F (41 o más). Un método con complejidad mayor a 10 es difícil de probar y de modificar.

Bash — medir antes y después

```
pip install radon pytest
radon cc -s -a src/          # complejidad ciclomática por método y promedio
radon raw -s src/            # líneas de código (SLOC)
radon mi -s src/             # índice de mantenibilidad
```

1.5 Patrones para sistemas con componentes de IA

Un modelo de lenguaje (LLM) es un componente **no determinista**: ante el mismo prompt puede responder distinto, puede equivocarse con seguridad, tiene costo por llamada y su proveedor puede cambiar. La literatura reciente de ingeniería de IA (Huyen, 2025; Iusztin y Labonne, 2024; Lakshmanan et al., 2020) y de código adaptable (McLean Hall, 2017) coincide en que los patrones clásicos son la forma de **aislar** ese componente:

Tabla 3. Patrones GoF aplicados a la integración de modelos de lenguaje

Necesidad al integrar IA	Patrón	Cómo ayuda
Cambiar de proveedor (Claude, OpenAI, modelo local) sin tocar el dominio	Strategy + Adapter	Una interfaz ProveedorLLM; cada proveedor es un adaptador intercambiable.
Probar sin pagar ni depender de la red	Strategy (doble de prueba)	En las pruebas se inyecta un LLM simulado con respuestas controladas.
Reducir costo y latencia	Decorator (caché)	Un envoltorio guarda respuestas a prompts repetidos.
Tolerar fallas del servicio	Decorator (reintentos, respaldo)	Reintenta o cambia a otro proveedor sin modificar el código cliente.
Evitar que una respuesta incorrecta llegue al usuario	Chain of Responsibility	Una cadena de guardrails valida la salida (precios inventados, afirmaciones no permitidas, longitud).

1.6 Flujo de trabajo y riesgos de usar IA para diseñar

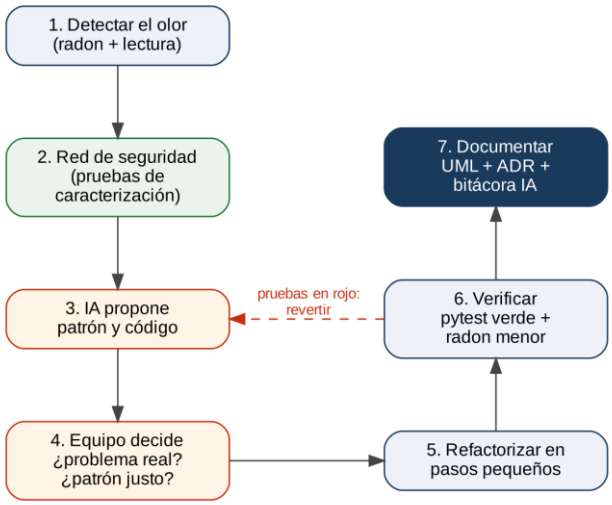


Figura 2. *Refactorización hacia patrones asistida por IA. Diagrama generado con Graphviz (DOT).*

Los asistentes de IA conocen muy bien el catálogo GoF, y ese es justamente su riesgo. Errores típicos que se deben vigilar:

- **"Patternitis" (sobreingeniería):** propone Abstract Factory para dos clases o un Visitor donde basta una función. Si el patrón no elimina un olor concreto, se rechaza.
- **Singleton por defecto:** lo sugiere para fábricas o configuraciones; introduce estado global y dificulta las pruebas. Es preferible inyectar la dependencia.
- **Cambios de comportamiento silenciosos:** al refactorizar puede alterar una regla (por ejemplo, reintentar también los pagos rechazados). Solo las pruebas lo revelan.

- **Código que "parece" correcto:** nombres de patrón en las clases, pero sin la estructura real (un "Strategy" con un if/elif dentro).

Regla de oro de esta práctica

La IA propone, el equipo decide y verifica.
Ningún patrón entra al código sin: (1) un olor concreto que lo justifique, (2) pruebas en verde antes y después, (3) una métrica que muestre la mejora y (4) un registro en la bitácora de IA.
Nunca ingrese datos personales, credenciales ni claves de API en un prompt.

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Caso: AgroConecta Majes (continuación de los Laboratorios 04 a 06). Se refactoriza el flujo **"confirmar y pagar un pedido"**, diseñado con UML en el Laboratorio 05, y se agrega una función de IA: **sugerir la descripción de un producto** con un modelo de lenguaje. Todo el código de esta sección se ejecutó; las métricas y los resultados de las pruebas son reales y están en el archivo de recursos.

Paso 1. Punto de partida: una versión que funciona, pero no escala

La primera versión se escribió rápido para cumplir el sprint. Los SDK de Yape y Plin (simulados en `externos.py`) tienen interfaces distintas, como ocurre en la realidad: uno cobra en soles y responde con un diccionario; el otro cobra en céntimos y responde con un código HTTP.

Python — src/pedidos/antes/servicio.py (método principal)

```
    def confirmar_y_pagar(self, pedido, medio):
        if pedido.estado != "BORRADOR":
            if pedido.estado == "CANCELADO":
                raise ValueError("El pedido está cancelado")
            elif pedido.estado == "PAGADO":
                raise ValueError("El pedido ya fue pagado")
            else:
                raise ValueError("Estado no válido")
        if not pedido.lineas:
            raise ValueError("El pedido no tiene líneas")
        total = 0
        for linea in pedido.lineas:
            total += linea.cantidad * linea.precio
        pedido.estado = "CONFIRMADO"
        aprobado, operacion = False, None
        if medio == "YAPE":
            for intento in range(3):
                try:
                    r = self.yape.realizar_cobro(pedido.celular, total)
                    aprobado, operacion = r["ok"], r.get("operacion")
                    break
                except TimeoutError:
                    if intento == 2:
                        aprobado = False
        elif medio == "PLIN":
            for intento in range(3):
                try:
                    codigo, tx = self.plin.charge(int(round(total * 100)), pedido.celular)
                    aprobado, operacion = codigo == 200, tx
                    break
                except TimeoutError:
                    if intento == 2:
                        aprobado = False
        else:
```

```
            raise ValueError(f"Medio de pago no soportado: {medio}")
        if aprobado:
            pedido.estado = "PAGADO"
            pedido.operacion = operacion
            self.eventos.append(f"whatsapp:{pedido.celular}:Pedido {pedido.id} pagado")
            self.eventos.append(f"reporte:{pedido.id}:{total:.2f}")
        else:
            pedido.estado = "CANCELADO"
        return pedido.estado
```

Salida de radon cc -s antes/servicio.py

```
antes/servicio.py
    M 32:4 ServicioPedidos.confirmar_y_pagar - C (15)
    C 24:0 ServicioPedidos - B (10)
    M 76:4 ServicioPedidos.sugerir_descripcion - B (7)
Average complexity: B (6.5)
```

Paso 2. Red de seguridad: pruebas de caracterización

Antes de tocar el código se escriben pruebas que **describen lo que el sistema hace hoy**. La técnica clave: **las mismas pruebas se ejecutan sobre la versión antes y la versión después** con un fixture parametrizado. Si todas pasan en ambas, el refactor conservó el comportamiento.

Python — tests/test_comportamiento.py (extracto)

```
@pytest.fixture(params=["antes.servicio", "despues.servicio"])
def m(request):
    return importlib.import_module(request.param)


def pedido(m, precio=3.5, cantidad=20):
    return m.Pedido(id=1, celular="959000111", lineas=[m.Linea("Leche fresca", cantidad, precio)])


def test_pago_yape_aprobado_publica_eventos(m):
    s = m.ServicioPedidos()
    assert s.confirmar_y_pagar(pedido(m), "YAPE") == "PAGADO"
    assert s.eventos == ["whatsapp:959000111:Pedido 1 pagado", "reporte:1:70.00"]

def test_reintenta_tras_dos_timeouts(m):
    yape = YapeSDK(fallas=2)
    assert m.ServicioPedidos(yape=yape).confirmar_y_pagar(pedido(m), "YAPE") == "PAGADO"
    assert yape.llamadas == 3
```

Se escribieron **12 pruebas de comportamiento**: pagos aprobados y rechazados, reintentos ante timeouts, transiciones inválidas, medio no soportado y guardrails de la IA.

Paso 3. Detectar olores y elegir patrones con IA

PROMPT IA 1 — Detector de olores

Actúa como revisor de código experto en refactorización (Fowler) y patrones GoF.

Contexto: [pegar antes/servicio.py y la salida de radon].

Tarea: identifica los olores de código. Para cada uno indica: líneas, olor, el patrón GoF que lo resolvería (o "ninguno" si no se justifica), el principio SOLID que mejora y el COSTO de aplicarlo.
Restricciones: no escribas código todavía; no propongas patrones sin un olor concreto; no uses Singleton.
Formato: tabla Markdown.

Tabla 4. Resultado del Prompt IA 1 y verificación del equipo

Olor detectado	Patrón propuesto por la IA	Decisión del equipo
if/elif por medio de pago que crece con cada medio nuevo	Strategy + Factory	Aceptado
El servicio conoce los detalles de cada SDK (soles/céntimos, dict/tupla)	Adapter	Aceptado
Bloque de reintentos duplicado para Yape y Plin	Decorator	Aceptado
Condicionales anidados sobre pedido.estado	State	Aceptado: coincide con la máquina de estados del Lab 05
El servicio arma a mano los mensajes de WhatsApp y de reportes	Observer	Aceptado
if/elif por proveedor de LLM y validaciones mezcladas	Strategy + Adapter + Chain of Responsibility	Aceptado
"FabricaPasarelas debería ser Singleton"	Singleton	Rechazado: estado global; impide inyectar SDK simulados en las pruebas
"Usar Abstract Factory para crear familias de pasarelas"	Abstract Factory	Rechazado: no hay familias de productos; sería sobreingeniería

Paso 4. Pagos: Adapter + Strategy + Decorator + Factory

Python — src/pedidos/pagos.py

```
from abc import ABC, abstractmethod
from dataclasses import dataclass


@dataclass(frozen=True)
class ResultadoPago:
    aprobado: bool
    operacion: str | None = None


class PasarelaPago(ABC):
    """Puerto (interfaz) común: todas las estrategias de pago se usan igual."""

    @abstractmethod
    def cobrar(self, monto: float, celular: str) -> ResultadoPago: ...


# ---------- ADAPTER: traduce cada SDK a la interfaz PasarelaPago ----------
class YapeAdapter(PasarelaPago):
    def __init__(self, sdk):
        self._sdk = sdk

    def cobrar(self, monto, celular):
```

```
        r = self._sdk.realizar_cobro(celular, monto)
        return ResultadoPago(r["ok"], r.get("operacion"))


class PlinAdapter(PasarelaPago):
    def __init__(self, cliente):
        self._cliente = cliente

    def cobrar(self, monto, celular):
        codigo, tx = self._cliente.charge(int(round(monto * 100)), celular)
        return ResultadoPago(codigo == 200, tx)



# ---------- DECORATOR: agrega reintentos sin modificar los adaptadores ----------
class ConReintentos(PasarelaPago):
    def __init__(self, interna: PasarelaPago, intentos: int = 3):
        self._interna, self._intentos = interna, intentos

    def cobrar(self, monto, celular):
        for _ in range(self._intentos):
            try:
                return self._interna.cobrar(monto, celular)
            except TimeoutError:
                continue
        return ResultadoPago(False)



# ---------- FACTORY (registro): crea la estrategia adecuada por nombre ----------
class FabricaPasarelas:
    def __init__(self):
        self._creadores = {}

    def registrar(self, medio: str, creador):
        self._creadores[medio] = creador

    def crear(self, medio: str) -> PasarelaPago:
        if medio not in self._creadores:
            raise ValueError(f"Medio de pago no soportado: {medio}")
        return ConReintentos(self._creadores[medio]())
```

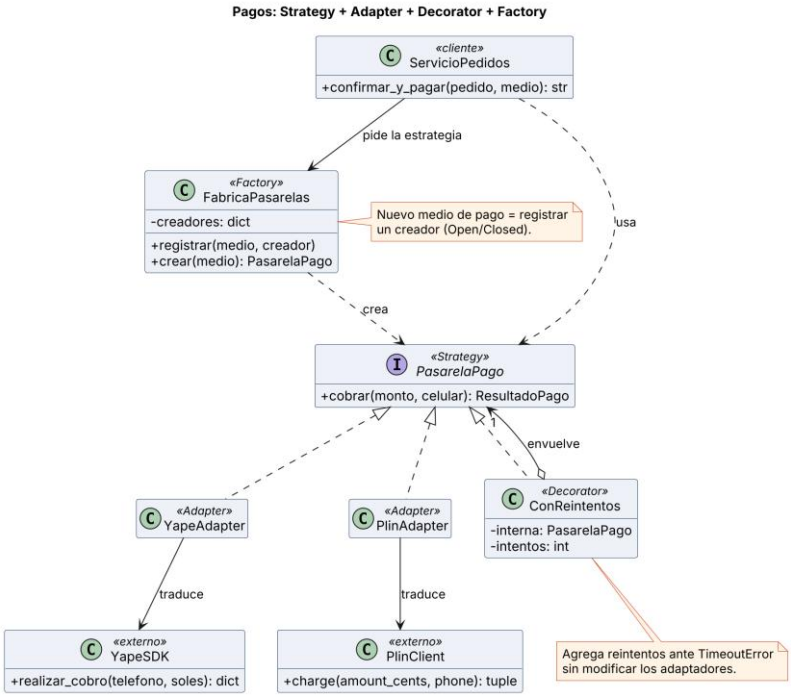


Figura 3. Diagrama de clases de los pagos (PlantUML): cuatro patrones colaborando.

Error de la IA detectado por las pruebas: la primera versión de `ConReintentos` generada por la IA reintentaba también cuando el pago era **rechazado**, lo que en producción podría generar cobros duplicados. La prueba `test_monto_sobre_limite_se_cancela_sin_eventos` falló y se corrigió: solo se reintenta ante `TimeoutError`.

Paso 5. Ciclo de vida del pedido: State

El patrón State implementa directamente la **máquina de estados del Laboratorio 05**. La clase base prohíbe todas las transiciones; cada estado redefine **solo** las permitidas, por lo que una transición inválida es imposible por construcción.

Python — src/pedidos/estados.py (extracto)

```
class EstadoPedido:
    nombre = "?"

    def confirmar(self, pedido):
        raise ValueError(f"No se puede confirmar un pedido {self.nombre}")

    def pagar(self, pedido, operacion):
        raise ValueError(f"No se puede pagar un pedido {self.nombre}")

    def cancelar(self, pedido):
        raise ValueError(f"No se puede cancelar un pedido {self.nombre}")


class Borrador(EstadoPedido):
    nombre = "BORRADOR"

    def confirmar(self, pedido):
        if not pedido.lineas:
            raise ValueError("El pedido no tiene líneas")
        pedido.cambiar_a(Confirmado())
```

```
    def cancelar(self, pedido):
        pedido.cambiar_a(Cancelado())


class Confirmado(EstadoPedido):
    nombre = "CONFIRMADO"

    def pagar(self, pedido, operacion):
        pedido.operacion = operacion
        pedido.cambiar_a(Pagado())

    def cancelar(self, pedido):
        pedido.cambiar_a(Cancelado())
```

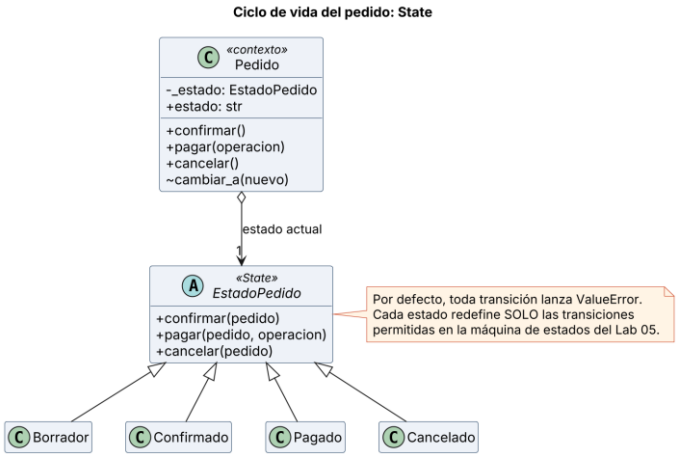


Figura 4. *Diagrama de clases del patrón State (PlantUML).*

Paso 6. Notificaciones: Observer

Python — src/pedidos/eventos.py (extracto)

```
class PublicadorEventos:
    def __init__(self):
        self._suscriptores = {}

    def suscribir(self, evento: str, suscriptor: Suscriptor):
        self._suscriptores.setdefault(evento, []).append(suscriptor)

    def publicar(self, evento: str, datos: dict):
        for s in self._suscriptores.get(evento, []):
            s.actualizar(evento, datos)


class NotificadorWhatsApp(Suscriptor):
    def __init__(self, bandeja):
        self._bandeja = bandeja

    def actualizar(self, evento, datos):
        self._bandeja.append(f"whatsapp:{datos['celular']}:Pedido {datos['id']} pagado")
```

Agregar un nuevo interesado (por ejemplo, una auditoría) **ya no requiere modificar el servicio**: basta suscribirlo. La prueba `test_observer_nuevo_suscriptor_sin_tocar_el_servicio` lo demuestra.

Paso 7. Aislar la IA: Strategy + Adapter + Decorator + Chain of Responsibility

Python — src/pedidos/ia.py (extracto: caché y guardrails)

```
class ConCache(ProveedorLLM):
    """DECORATOR: evita pagar dos veces por el mismo prompt."""

    def __init__(self, interno: ProveedorLLM):
        self._interno, self._cache = interno, {}

    def completar(self, prompt):
        if prompt not in self._cache:
            self._cache[prompt] = self._interno.completar(prompt)
        return self._cache[prompt]


# ---------- CHAIN OF RESPONSIBILITY: guardrails de salida ----------
class Guardrail(ABC):
    def __init__(self, siguiente=None):
        self._siguiente = siguiente

    def validar(self, texto: str) -> str:
        texto = self._aplicar(texto)
        return self._siguiente.validar(texto) if self._siguiente else texto

    @abstractmethod
    def _aplicar(self, texto: str) -> str: ...


class SinPrecios(Guardrail):
    def _aplicar(self, texto):
        if "S/" in texto:
            raise ValueError("Guardrail: la IA incluyó un precio")
        return texto


class SinAfirmacionesProhibidas(Guardrail):
    PROHIBIDAS = ["orgánico certificado", "cura", "garantizado"]

    def _aplicar(self, texto):
        for palabra in self.PROHIBIDAS:
            if palabra in texto.lower():
                raise ValueError(f"Guardrail: afirmación no permitida '{palabra}'")
        return texto
```

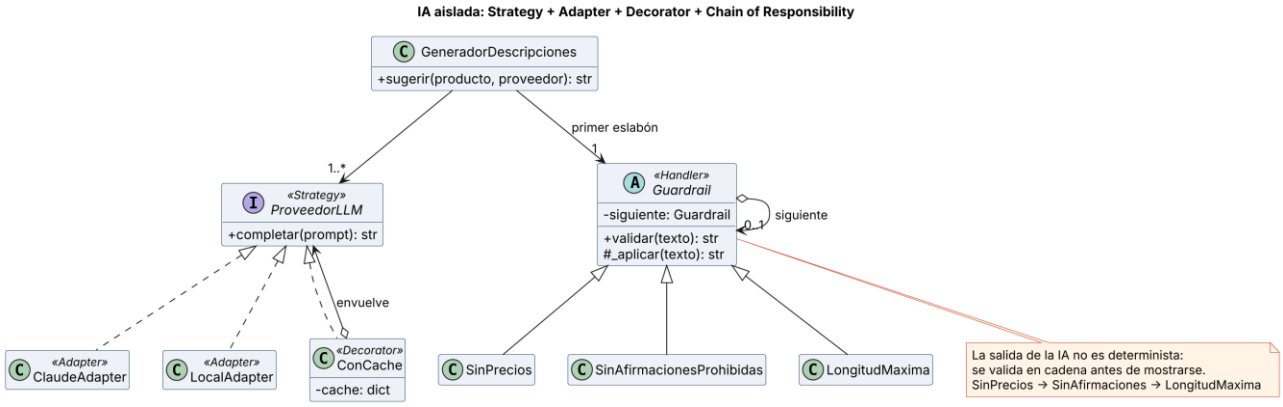


Figura 5. *El componente de IA aislado detrás de cuatro patrones (PlantUML).*

¿Por qué importa? La respuesta del LLM se trata como **dato no confiable**: puede inventar un precio ("a solo S/ 2.50") o una afirmación de salud ("cura la gastritis"). La cadena de guardrails la bloquea antes de que llegue al comprador. Como el proveedor es una Strategy, en las pruebas se inyecta un `ClienteLLMFalso` con la respuesta exacta que se quiere probar: **no se necesita clave de API ni conexión**.

Paso 8. El servicio después del refactor

Python — src/pedidos/servicio.py (método principal)

```
    def confirmar_y_pagar(self, pedido: Pedido, medio: str) -> str:
        pasarela = self.fabrica.crear(medio)
        pedido.confirmar()
        resultado = pasarela.cobrar(pedido.total(), pedido.celular)
        if resultado.aprobado:
            pedido.pagar(resultado.operacion)
            self.publicador.publicar("pedido_pagado", {
                "id": pedido.id, "celular": pedido.celular, "total": pedido.total()})
        else:
            pedido.cancelar()
        return pedido.estado
```

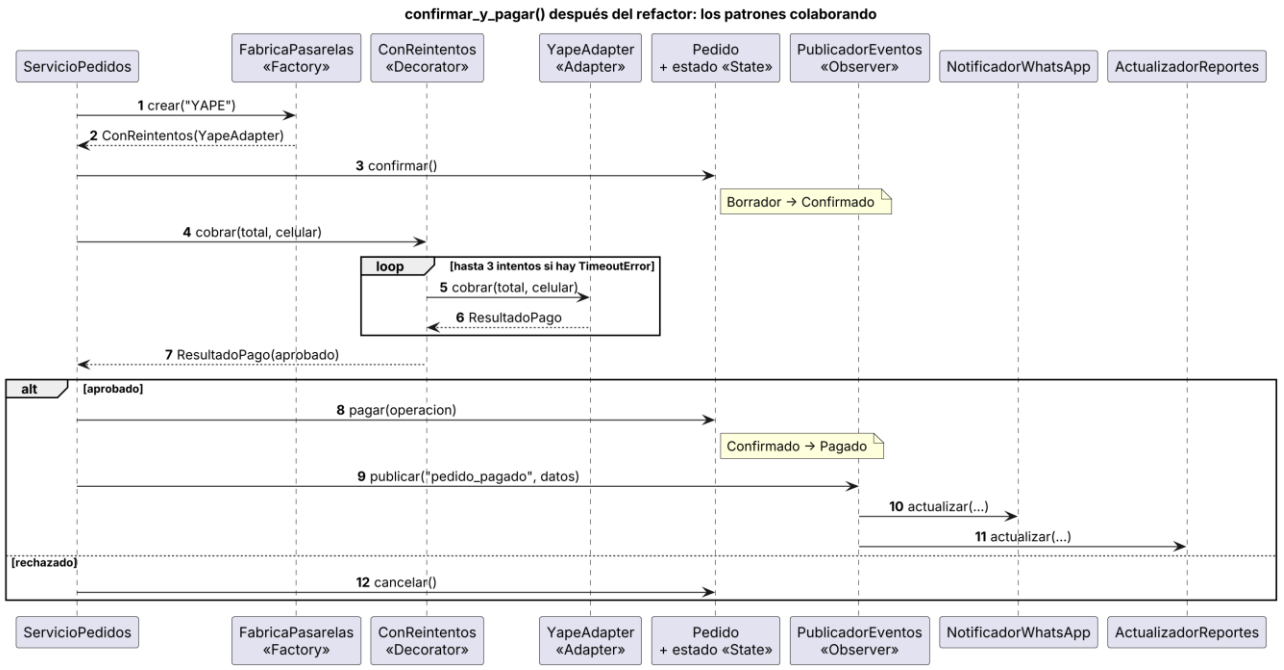


Figura 6. *Diagrama de secuencia (PlantUML): los patrones colaborando en confirmar_y_pagar().*

Paso 9. Verificar: pruebas y métricas

Salida de pytest -v (resumen)

```
test_comportamiento.py::test_pago_yape_aprobado_publica_eventos[antes.servicio]   PASSED
test_comportamiento.py::test_pago_yape_aprobado_publica_eventos[despues.servicio] PASSED
...
test_comportamiento.py::test_medio_no_soportado_no_altera_el_estado[antes.servicio]   XFAIL
test_comportamiento.py::test_medio_no_soportado_no_altera_el_estado[despues.servicio] PASSED
...
test_patrones.py::test_open_closed_nuevo_medio_sin_modificar_el_servicio PASSED
test_patrones.py::test_state_impide_transiciones_invalidas PASSED
test_patrones.py::test_decorator_cache_evita_llamadas_repetidas PASSED
test_patrones.py::test_observer_nuevo_suscriptor_sin_tocar_el_servicio PASSED
======================== 27 passed, 1 xfailed in 0.04s ========================
```

El refactor encontró un error: en la versión antes, si el medio de pago no existía, el pedido quedaba en estado CONFIRMADO aunque se lanzara la excepción. Con la Factory, el medio se valida **antes** de cambiar el estado. La prueba se marcó como `xfail` (falla esperada) en la versión antes para documentar el error.

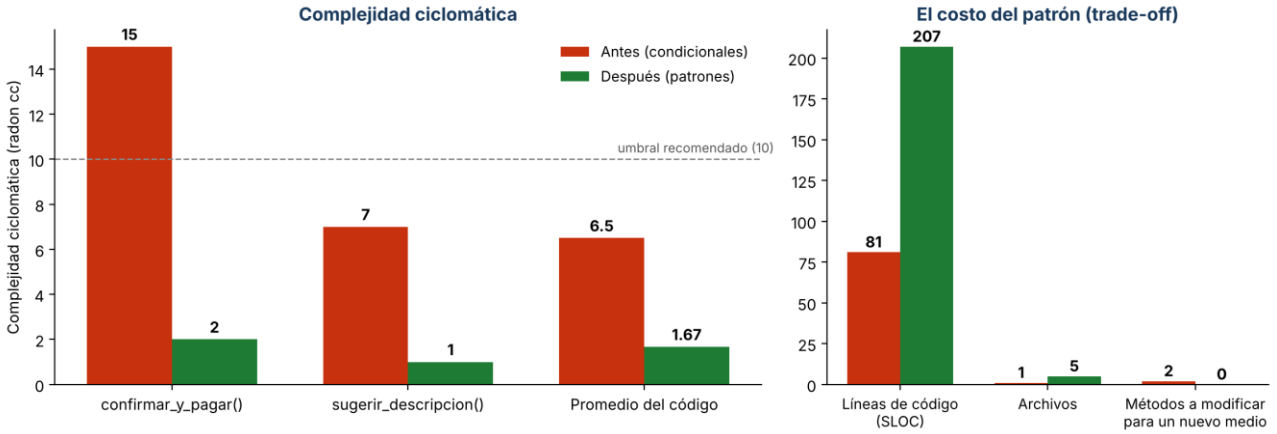


Figura 7. *Métricas reales (radon 6.0.1): la complejidad baja de forma drástica, pero el código crece. Gráfico elaborado con matplotlib.*

Lectura honesta del trade-off: el método principal pasó de complejidad **15 (C) a 2 (A)** y el promedio de **6,5 a 1,67**; agregar un medio de pago ya no exige modificar el servicio. A cambio, el código pasó de **81 a 207 líneas** y de **1 a 5 archivos**. Si AgroConecta solo fuera a tener un medio de pago y nunca IA, la versión antes habría sido razonable. Los patrones se justifican por los **cambios previstos**, no por sí mismos.

Paso 10. Bitácora de IA y ADR

Tabla 5. Extracto de bitacora-ia.md

#	Prompt (resumen)	Propuesta de la IA	Verificación	Decisión
1	Detector de olores (Prompt 1)	8 olores y patrones	Revisión con radon y lectura del código	6 aceptados, 2 rechazados
2	Refactor solo de pagos (Prompt 2)	ConReintentos que reintenta todo	Prueba de monto sobre límite en rojo	Corregida
3	Refactor de estados	Tabla de transiciones con diccionario	Válida, pero los estados tendrán comportamiento propio (reembolso)	Rechazada a favor de State
4	Guardrails de IA	Usar otro LLM como juez para validar	Costo y latencia extra para un MVP de 1 mes	Rechazada; reglas deterministas
5	Diagramas PlantUML desde el código	Diagramas de clases y secuencia	Comparación con el código real (pyreverse)	Aceptada con correcciones de nombres

Markdown — docs/architecture/adr/005-patrones-de-diseno.md

```
# ADR-005: Patrones de diseño en el módulo Pedidos
- **Estado:** Aceptado (21/10/2026)
## Contexto
confirmar_y_pagar() tenía complejidad 15; cada medio de pago o proveedor de IA nuevo obligaba
a modificar el servicio. La salida del LLM no es confiable (puede inventar precios).
## Decisión
Adapter + Strategy + Factory para pagos; Decorator para reintentos y caché; State para el ciclo
de vida (Lab 05); Observer para notificaciones; Chain of Responsibility para guardrails de IA.
Se rechazan Singleton y Abstract Factory (ver bitácora, entrada 1).
## Consecuencias
(+) Complejidad máxima 5; nuevos medios y proveedores sin modificar el servicio; IA probable sin API.
(-) 207 líneas frente a 81 y más indirección: el equipo debe conocer los patrones.
```

Paso 11. Estructura del repositorio y contenido de cada archivo

Se trabaja en el **mismo repositorio** de los laboratorios anteriores. Los códigos **E1 a E8** son los **entregables** de los ejercicios propuestos (E1 = Ejercicio 1, etc.).

Estructura del repositorio cs-2026b-lab04-grupoXX (continuación)

```
cs-2026b-lab04-grupoXX/
├── README.md                          # E8: sección "Patrones de diseño (Lab 07)"
├── src/<modulo>/
│   ├── antes/servicio.py              # E1: versión ingenua (se conserva para comparar)
│   ├── externos.py                    # E1: servicios externos simulados
│   ├── servicio.py, pagos.py, ...     # E2–E5: versión con patrones
│   └── ia.py                          # E5: componente de IA aislado
├── tests/
│   ├── test_comportamiento.py         # E1: pruebas de caracterización (antes y después)
│   └── test_patrones.py               # E2–E5: pruebas que demuestran cada patrón
└── docs/
    ├── architecture/adr/005-patrones-de-diseno.md   # E8
    └── design/patrones/
        ├── olores.md                  # E1: olores detectados y patrón propuesto
        ├── metricas.md                # E6: radon antes/después y trade-off
        ├── <patron>.puml + img/       # E7: un diagrama por patrón aplicado
        └── bitacora-ia.md             # E7: interacciones con la IA
```

Tabla 6. Qué va dentro de cada archivo

Código	Archivo	Contenido mínimo obligatorio	Modelo
E1	antes/, externos.py, test_comportamiento.py, olores.md	Versión ingenua del flujo crítico; 8 o más pruebas de caracterización en verde; salida de radon; tabla de 4 o más olores.	Pasos 1–3
E2	pagos.py (o equivalente), test_patrones.py	Strategy + Adapter + Factory para la variación principal de su caso; prueba de extensión sin modificación.	Paso 4
E3	estados.py	State que implemente la máquina de estados del Lab 05; prueba de transición inválida.	Paso 5
E4	eventos.py y decoradores	Observer y Decorator (o el patrón sugerido para su caso), cada uno con una prueba.	Pasos 4 y 6
E5	ia.py	Proveedor de IA como Strategy/Adapter, caché (Decorator), 2 o más guardrails en cadena y LLM simulado en las pruebas.	Paso 7
E6	metricas.md	Tabla de radon antes/después, SLOC, análisis del trade-off y 1 o más patrones rechazados con justificación.	Paso 9
E7	*.puml, img/, bitacora-ia.md	Un diagrama PlantUML por patrón, clases.puml del Lab 05 actualizado y 5 o más interacciones con IA.	Pasos 4–10
E8	ADR-005, README.md, tag	ADR de los patrones, sección en el README, Pull Request revisado y etiqueta v0.7-patrones.	Paso 10

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Organización: se mantienen los **mismos grupos, casos y repositorio** de los Laboratorios 04 a 06. Se refactoriza el **flujo crítico** que modelaron en el diagrama de secuencia del Laboratorio 05, en Python o Java (en Java use JUnit y la métrica de complejidad de PMD o SonarLint).

Condiciones comunes

- Plazo: el MVP debe estar en producción en 1 mes; aplique solo los patrones que su caso necesita.
- Mínimo 4 patrones GoF distintos en total, cada uno justificado por un olor y demostrado por una prueba.
- Sin claves de API en el repositorio: los LLM y servicios externos se simulan en las pruebas.
- Regla de oro: la IA propone, el equipo decide y verifica; todo se registra en bitacora-ia.md.

Patrones sugeridos y función de IA para cada caso

Tabla 7. Sugerencias por caso (validar contra los olores reales de su código)

N.º	Caso	Variación o problema → patrón sugerido	Función de IA a aislar (E5)
1	CitaSalud Arequipa	Política de asignación de cupos → Strategy; ciclo de la cita → State; recordatorios → Observer; validaciones de reserva → Chain	Clasificar el motivo de consulta para sugerir la especialidad (sin diagnosticar)
2	PicanteríaYa	Asignación de repartidor (más cercano o menos carga) → Strategy; pasarelas y mapas → Adapter; seguimiento → Observer	Redactar la descripción del plato del día
3	AireMisti IoT	Sensores de distintas marcas → Adapter; índice de calidad (OMS o ECA peruano) → Strategy; alertas → Observer; filtro de lecturas anómalas → Decorator	Explicar una alerta en lenguaje sencillo para la ciudadanía
4	BiblioUNSA	Sistema académico → Adapter; multa según tipo de usuario → Strategy; préstamo → State; caché de consultas → Proxy	Recomendar libros similares del catálogo
5	VotoEPIS	Voto como acción auditable → Command; elegibilidad → Chain; elección → State; auditoría → Observer	Asistente de preguntas sobre el reglamento (la IA NO participa en el conteo)
6	RutaSIT Arequipa	Algoritmo de tiempo de llegada → Strategy; GPS de distintos operadores → Adapter; suscripción a paraderos → Observer	Explicar el motivo de un retraso a partir de los eventos
7	San Camilo en Línea	Pedido de varios puestos → Composite; tarifa de delivery → Strategy; avisos a comerciantes → Observer	Normalizar nombres de productos ("papa amarilla", "papa amarila")
8	SismoReporta AQP	Reporte sin señal → Command + cola; validación y priorización → Chain; reporte → State	Clasificar la severidad del daño a partir de la descripción
9	ChacraSmart Majes	Válvula con falla segura → State; orden de riego → Command; política de riego → Strategy; sensores → Observer	Explicar la recomendación de riego al agricultor
10	EcoRecicla AQP	Reglas de puntos → Strategy; campañas y bonificaciones → Decorator; solicitudes → State; avisos → Observer	Clasificar el tipo de residuo a partir de la descripción

Parte A — Sesión de laboratorio (90 minutos)

Tiempo	Actividad	Producto / checkpoint
0–10 min	Presentación del tema; instalar pytest y radon	`pytest` y `radon` funcionando sobre el ejemplo
10–25 min	Demostración del docente: Pasos 2, 3, 4 y 9	Ejemplo ejecutado en cada equipo (27 passed, 1 xfailed)
25–45 min	E1. Versión antes, pruebas de caracterización y olores	test_comportamiento.py en verde y olores.md con commit
45–65 min	E2. Primer refactor: Strategy + Adapter + Factory	Pruebas en verde sobre antes y después
65–80 min	E3. State para la entidad principal	Prueba de transición inválida en verde

80–90 min	Checkpoint	El docente revisa pytest, radon y olores.md con la lista de cotejo

E1. Versión antes, red de seguridad y olores (20 min)

1. Implemente (o genere con IA y revise) una **versión ingenua** del flujo crítico de su caso en `src/<modulo>/antes/servicio.py`, usando el diagrama de secuencia del Lab 05. Simule los servicios externos en `externos.py` con interfaces distintas entre sí.
2. Escriba **8 o más pruebas de caracterización** en `tests/test_comportamiento.py` con el fixture parametrizado del Paso 2 (casos de éxito, error, límites y reintentos). Todas deben pasar.
3. Ejecute `radon cc -s -a` y guarde la salida.
4. Use el **Prompt IA 1** y registre en `olores.md` **4 o más olores** con su patrón propuesto, el principio SOLID y la decisión del equipo. Al menos una propuesta de la IA debe quedar **rechazada** con justificación.
5. Haga commit con el mensaje `E1: version antes y pruebas de caracterizacion`.

☐ **Lista de verificación:** las pruebas verifican comportamiento observable (estado, eventos, excepciones), no detalles internos; ninguna prueba llama a servicios reales.

E2. Strategy + Adapter + Factory (20 min)

1. Identifique la **variación principal** de su caso (columna 3 de la tabla) y cree la interfaz común (puerto).
2. Escriba un **Adapter** por cada servicio externo simulado y una **Factory** (registro) que cree la estrategia adecuada.
3. Pida a la IA el refactor **solo de este patrón** (Prompt IA 2, a continuación): "aplica únicamente Strategy + Adapter + Factory; no cambies el comportamiento observable".
4. Agregue el fixture `despues.servicio` a sus pruebas: **todas** deben pasar en ambas versiones.
5. Escriba una prueba que agregue una variante nueva **sin modificar el servicio** (como `test_open_closed_...`). Commit: `E2: strategy adapter factory`.

☐ **Lista de verificación:** el servicio ya no contiene if/elif por variante; los adaptadores no contienen reglas de negocio.

PROMPT IA 2 — Refactor de un solo patrón

Aplica ÚNICAMENTE el patrón [Strategy + Adapter + Factory] al siguiente código [pegar].
Reglas: (1) no cambies el comportamiento observable; las pruebas [pegar test_comportamiento.py] deben seguir pasando;
(2) no introduzcas Singleton ni estado global; (3) no agregues patrones que no te pedí;
(4) muestra los archivos completos y explica en 3 líneas qué olor elimina.

E3. State para la entidad principal (15 min)

1. Tome la máquina de estados del Laboratorio 05 (`estados-<entidad>.mmd`).
2. Cree una clase base que **prohíba todas las transiciones** y una subclase por estado que redefina solo las permitidas.
3. Reemplace los condicionales sobre el atributo de estado por delegación al objeto estado. Escriba una prueba de transición inválida. Commit: `E3: state`.

☐ **Lista de verificación:** cada transición del diagrama de estados tiene un método; ninguna clase de estado tiene if sobre el nombre de otro estado.

Parte B — Trabajo autónomo (20/10/2026 al 24/10/2026)

E4. Observer y Decorator (o el patrón sugerido para su caso)

1. Reemplace las notificaciones "a mano" por un **Observer** con al menos 2 suscriptores.
2. Extraiga una responsabilidad transversal (reintentos, registro, caché, filtro) a un **Decorator**.
3. Si su caso sugiere otro patrón (Composite, Command, Proxy), puede usarlo en lugar de uno de estos dos. Cada patrón necesita **una prueba que lo demuestre**.

E5. Aislar el componente de IA

1. Implemente la función de IA de su caso (columna 4) detrás de una interfaz `ProveedorLLM` con **al menos un adaptador**.
2. Agregue un **Decorator de caché** y una **cadena de 2 o más guardrails** adecuados a su dominio (por ejemplo, CitaSalud: "no emitir diagnósticos"; AireMisti: "no inventar valores de PM10").
3. Pruebe con un **LLM simulado** que devuelva respuestas válidas e inválidas: los guardrails deben bloquear las inválidas.
4. **Opcional:** conecte un proveedor real usando una variable de entorno; nunca suba la clave al repositorio (agregue `.env` al `.gitignore`).

E6. Métricas y trade-off

1. Complete `metricas.md` con una tabla de radon (complejidad por método y promedio) antes y después, y las líneas de código (SLOC).
2. Responda en 5 a 8 líneas: ¿qué ganaron, qué costó y en qué situación la versión antes habría sido suficiente?
3. Documente **al menos un patrón que decidieron NO aplicar** y por qué.

E7. Diagramas como código y bitácora

1. Elabore un diagrama de clases PlantUML **por cada patrón** aplicado, con los estereotipos del rol (<<Strategy>>, <<Adapter>>, etc.), como en las Figuras 3 a 5.
2. Actualice `docs/design/clases.puml` del Laboratorio 05 y verifique con `pyreverse` que el diagrama coincide con el código.
3. Complete `bitacora-ia.md` con **5 o más interacciones**; al menos una debe ser una corrección detectada por las pruebas.

E8. ADR, README, revisión cruzada y entrega

1. Redacte `ADR-005` (patrones adoptados y rechazados) con el formato del Laboratorio 04.
2. Agregue al `README.md` la sección **"Patrones de diseño (Lab 07)"** con la tabla olor → patrón → prueba y la tabla de métricas.
3. Cada integrante abre un **Pull Request** con uno de sus patrones; otro integrante lo revisa respondiendo: ¿qué olor elimina?, ¿hay una prueba?, ¿es el patrón más simple que resuelve el problema?
4. Cree la etiqueta `v0.7-patrones` y entregue el enlace del repositorio en el Aula Virtual hasta el **sábado 24/10/2026 a las 23:59**.

Reto opcional (+2 puntos, sin exceder la nota máxima)

```
Opción A — "Kata de patrones con IA adversaria": pida a la IA que escriba una versión de su flujo con
la
máxima cantidad de patrones posible ("patternitis"). Mida con radon y SLOC las tres versiones (antes,
la suya
y la sobrediseñada) y argumente en el README por qué la suya es la adecuada.
```

```
Opción B — Integración continua: configure una GitHub Action que ejecute pytest y falle si algún
método
supera complejidad 10 (radon cc --min C).
```

Cronograma de la semana

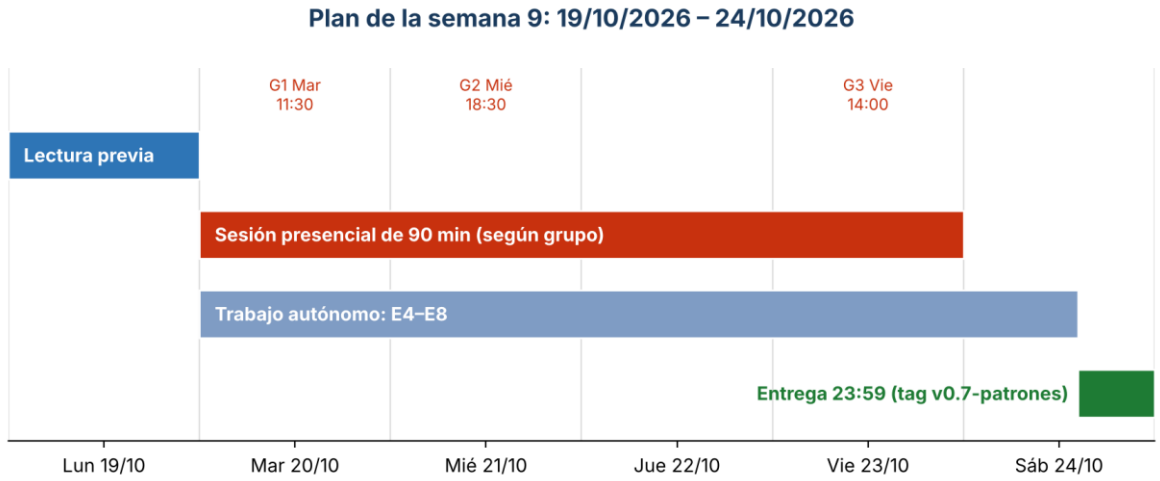


Figura 8. *Plan de trabajo de la semana (19/10/2026 – 24/10/2026).*

Fecha	Actividad
Lun 19/10/2026	Publicación de la guía. Lectura previa: marco conceptual y refactoring.guru (Strategy, Adapter, State y Observer).
Mar 20/10 – Vie 23/10	Sesión de 90 min según el horario del grupo (Parte A: E1–E3) y checkpoint.
Día siguiente a la sesión	E4 (Observer y Decorator) y E5 (IA aislada).
Antes del sábado	E6 (métricas) y E7 (diagramas y bitácora). Revisión cruzada por Pull Request.
Sáb 24/10/2026	E8: ADR-005, README, etiqueta v0.7-patrones y entrega en el Aula Virtual hasta las 23:59.

IV. CUESTIONARIO

1. ¿Qué diferencia hay entre un patrón arquitectónico (Laboratorio 04) y un patrón de diseño? Dé un ejemplo de cada uno en su caso.
2. Explique con su código cómo Strategy materializa el principio abierto/cerrado. ¿Qué tuvo que modificar para agregar una variante nueva?
3. ¿En qué se diferencian Adapter y Decorator si ambos "envuelven" un objeto?
4. ¿Por qué el patrón State es la implementación natural de un diagrama de máquina de estados? ¿Qué ganó frente a usar condicionales?
5. ¿Por qué son indispensables las pruebas de caracterización antes de refactorizar? Describa un error que sus pruebas detectaron.
6. ¿Qué patrón propuso la IA que ustedes rechazaron y por qué? ¿Qué es la "patternitis"?
7. ¿Por qué se debe tratar la salida de un LLM como dato no confiable? Describa los guardrails de su caso y qué protegen.

8. Según sus métricas, ¿valió la pena el refactor? Argumente con la complejidad ciclomática, las líneas de código y los cambios previstos.

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Freeman, E., & Robson, E. (2020). *Head first design patterns* (2nd ed.). O'Reilly Media.

Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2nd ed.). Addison-Wesley.

Kerievsky, J. (2004). *Refactoring to patterns*. Addison-Wesley.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

McLean Hall, G. (2017). *Adaptive code: Agile coding with design patterns and SOLID principles* (2nd ed.). Microsoft Press.

Lakshmanan, V., Robinson, S., & Munn, M. (2020). *Machine learning design patterns*. O'Reilly Media.

Huyen, C. (2025). *AI engineering: Building applications with foundation models*. O'Reilly Media.

Iusztin, P., & Labonne, M. (2024). *LLM engineer's handbook*. Packt Publishing.

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering, SE-2*(4), 308–320.

Valente, M. T. (2020). *Engenharia de Software Moderna*, cap. 6: Padrões de projeto. https://engsoftmoderna.info/cap6.html

Refactoring.Guru. (s. f.). *Patrones de diseño*. https://refactoring.guru/es/design-patterns

Radon. (s. f.). *Documentación oficial*. https://radon.readthedocs.io/

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

- Observación sistemática en el laboratorio (checkpoint E1–E3).
- Análisis de productos: código, pruebas, métricas y diagramas del repositorio.
- Revisión entre pares mediante Pull Requests.

INSTRUMENTOS:

- Lista de cotejo del checkpoint.
- Rúbrica analítica (criterios abajo).
- Resultados de pytest y radon.
- Historial de commits y etiqueta v0.7-patrones.

CRITERIOS DE EVALUACIÓN

Criterio	Descripción	Puntaje
E1 Red de seguridad y olores	Versión antes, 8 o más pruebas de caracterización y 4 o más olores con decisión.	3
E2–E4 Patrones aplicados	4 o más patrones GoF bien implementados, cada uno justificado por un olor y con su prueba.	6
E5 IA aislada	Proveedor intercambiable, caché, 2 o más guardrails y pruebas con LLM simulado.	3
E6 Métricas y trade-off	radon y SLOC antes/después; análisis honesto; patrón rechazado justificado.	2
E7 Diagramas y bitácora	Un diagrama por patrón, clases.puml actualizado y bitácora con 5 o más interacciones.	3
E8 ADR, README y entrega	ADR-005, README, Pull Requests revisados y etiqueta v0.7-patrones a tiempo.	3

Total	Reto opcional (kata adversaria o integración continua): +2 puntos (nota máxima 20).	20